

Experiment No: 2

SHELL PROGRAMMING

2.a) Aim: To write a shell program to check whether the given number is odd or even

Algorithm

Step 1: Start

Step 2: Read num

Step 3: If Num % 2 == 0 go to Step 4 else Step 5

Step 4: Display even and Stop

Step 5: Display odd and Stop

Result

The program was executed successfully and result was verified.

2.1)

Aim

To write a shell program to find largest among 3 numbers

Algorithm

Step 1: Start

Step 2: Read 3 values a, b, c

Step 3: if $a > b, c$. a is largest
otherwise go to step 6.

Step 4: if $b > a, c$ b is largest
otherwise go to step 6.

Step 5: if neither a or b is largest,
 c should be largest.

Step 6: Display the largest sum

Step 7: Stop

Result

The program was executed successfully and the result was verified.

2.c) Aim

To write a shell program to calculate the sum of digits of a number.

Algorithm

step 1: start.

step 2: Read n .

step 3: Declare $Sum = 0$

step 4: $rem = n \% 10$

4.1 $Sum = Sum + rem$

4.2 $n = n / 10$.

step 5: if $(n > 0)$ then go to step 4 else go to step 6.

step 6: Print Sum

step 7: stop.

Result

The program was executed successfully and result was verified.

2.d) Aim

To write a shell programming to print the fibonacci series

Algorithm

step1: start

step2: Read the limit n from the user

step3: Initialise $a_1 = 0$, $a_2 = 1$ and $b = 0$

step4: Display a_1 and a_2

step5: While n is not equal to b do

5.1 let $a_3 = a_1 + a_2$

5.2 Display a_3

5.3 let $a_1 = a_2$

5.4 let $a_2 = a_3$

5.5 let $b = b + 1$

step6: Stop

Result

The program was executed successfully and result was verified.

2.e) Aim

To write a shell program to perform
Simple calculator.

Algorithm

Step 1: start.

Step 2: Read 2 number a, b and choice ch.

Step 3: if ch is 1, add 2 numbers and
print result.

Step 4: if ch is 2, subtract 2 numbers
and print result.

Step 5: If ch is 3, multiply 2 numbers
and print result.

Step 6: If ch is 4, divide 2 numbers
and print the result.

Step 7: If the operator is not one of
above print an error message.

Step 8: stop.

Result

The program was executed successfully and
the output was verified.

~~huz/m~~

Experiment No. 3

SYSTEM CALLS OF LINUX
OPERATING SYSTEM.

3.a) AIM

To write a new process using `fork()` system call.

THEORY

`fork()`: Creates a new copy called the child out of original process that is called parent.

Syn: Var-name = `fork()`

`getpid()`: returns process ID of the calling process.
Syn: `pid_t getpid(void)`

`waitpid()`: Suspends execution of the current process until a child specified by `pid` argument has changed state.

`wait()`: Suspends execution of current process until one of its children terminates.

`sleep()`: Causes the calling thread sleep either until the number of rest time seconds specified in seconds.

have dispersed or until a signal arrived which is not ignored.

exit() : causes the shell or program to terminate.

syn: `exit(n)`

ALGORITHM

Step 1: Start.

Step 2: Declare Variable `v`

Step 3: Call the System call `Fork V = Fork(c)`

Step 4: Fork returns a value to `v`

Step 5: if `V = -1` display 'error'

ie, the process ^{creation} ~~was~~ was unsuccessful.

Step 6: if `V > 0`, a parent process is created and display the process id using `getpid()` Function.

Step 7: else (`V = 0`), a child process is created and display the process id using `getpid()` Function.

Step 8: Stop.

RESULT

The program was executed successfully and output is verified.

Page 7/103/24

3.6) Aim

To get the details of a file using `readdir()` system call.

Theory

`opendir()` : Open a directory stream corresponding to the directory named by the filename argument

`readdir()` : returns a pointer to a direct structure representing the next directory entry in the directory stream pointed to by `dir`.

ALGORITHM

Step 1: Start.

Step 2: Declare struct `dirent`

Step 3: Declare a character name[16] of size 16.

Step 4: Declare `DIR * dirptr`.

Step 5: Read the directory name & file name into name using Scanner.

Step 6: Use `dirptr = opendir(name)` system call to open the directory.

Step 7: if `dirptr == NULL`, print "Error"

Step 8: Use

while (ptr = readdir (dirptr))

using while loop read the files inside the directory.

Step 9: Display the name of the file

using ptr → d_name.

Step 10: Display the type of the file using

ptr → d_type.

Step 11: Display the File number of the file

using ptr → d_fileno.

Step 12: Stop

RESULT

The program was executed Successfully and output was verified.

Supriya
7/10/24

Experiment No. 4

PROCESS SCHEDULING.

4.a) AIM

To write a C program to perform FCF process scheduling.

ALGORITHM.

Step 1: Start.

Step 2: Declare a structure process with attributes Pid, bt, wt and tat.

Step 3: Declare variable i and p, let $avgw = 0$ and $avgt = 0$ and p of size 10.

Step 4: Read the number of process m.

Step 5: Read the process details (process id and burst time) for each process and store them in p.

Step 6: Read the arrival time for all processes.

Step 7: Sort the processes with respect to the arrival time.

Step 8: If the process is the first one then its waiting time = 0.

Step 9: else

Turnaround time = completion time of a process - ~~burst~~ Arrival time

• $\text{Waiting time} = \text{Turnaround time} - \text{burst time}$.

Step 10: Calculate the average waiting time and average turnaround time for all the processes.

Step 11: Display process id, burst time, waiting time and turnaround time along with the average waiting time and turnaround time for all the processes.

Step 12: Stop

Result.

The program was executed successfully and output was verified.

Bye
28/03/24

4.6)

AIM

To write a program to perform SJF process scheduling.

ALGORITHM

Step 1: Start.

Step 2: Declare a structure process with the attributes pid, wt, bt and tat.

Step 3: Declare variables $i, n, \text{avgw} = 0, \text{avgt} = 0, j$ and p of size 10.

Step 4: Read the number of processes n .

Step 5: Read the process id, burst time for each process and store in p .

Step 6: Set all the process arrival time $= 0$.

Step 7: Sort the array p based on the burst time of the processes from $i = 0$ to n and from $j = 0$ to $n - i - 1$ if burst time of $j >$ burst time of $j + 1$ swap $p[j + 1]$ and $p[j]$

Step 8: Calculate the waiting time and turnaround time.

If the process is the first one, waiting time $= 0$.

else

Turn around time = completion of time of the process - Arrival time.

Waiting time = Turnaround time -

Burst time.

Step 9: Calculate the average waiting time and turnaround time for all the processes.

Step 10: ~~Calculate~~^{Display} average waiting time, average turn around time, process id, burst time, waiting time and turnaround time

Step 11: Stop.

RESULT

The program was executed successfully and the output was verified.

By 20/03/24

u.c)

AIM

To write a program to perform priority scheduling.

ALGORITHM

Step 1: Start.

Step 2: Declare a structure process with the attributes pid, bt, wt, tat.

Step 3: Declare Variable i, n, avgw = 0, avgt = 0, j and p of size 10.

Step 4: Read the number of process n.

Step 5: Read the process id, burst time, priority for each process and store them in p.

Step 6: Set the arrival time of all processes as 0.

Step 7: Sort the array p based on the priority of the processes.

From i = 0 to n and from j = 0 to n-1 - if priority of j > priority of j+1
Swap p[j+1] and p[j]

Step 8: Calculate waiting time and turnaround time.

If the process is the first one, waiting time is zero.

waiting time = Turnaround time - burst time.

Turnaround time = completion time - arrival time.

Step 9: Calculate average turnaround time and average waiting time.

Step 10: Display the process id, burst time, waiting time, turnaround time and the average waiting time and average turnaround time.

Step 11: Stop.

RESULT

The program was executed successfully and output was verified.

By 26/03/24

4.d) AIM

To write a program to perform round robin scheduling.

ALGORITHM

tbt - copy of burst time

x - At present, which time we are in.
initially $x=0$.

y - Indicates how many process has completed its execution.

i - indicates at present which process going to cpu

ntq - indicates number of time quantum.

Step 1: Start.

Step 2: Declare a Structure process with the attributes bt, wt, tat, at, ct, tbt.

Step 3: Declare variable n, $i=0$, j, tq, $x=0$, $y=0$, $ntq=0$, b[20], c[20], avgwt, avgtat.

Step 4: Read number of process n and time quantum tq.

Step 5: Read process id, burst time, set arrival time to 0 and copy burst time to tbt of each process and store them in Pr.

Step 6: Loop through each processing until each process completed.

6.1 if burst time of $Pr[i] \neq 0$

6.1.1 if burst time of $pr[i] > \text{time quantum}$

a) x increments by tq .

b) burst time of $pr[i] = \text{bursttime of } pr[i] - tq$.

6.1.2 otherwise

a) x incremented by burst time of $pr[i]$

b) burst time of $pr[i] = 0$ then ~~one~~ process i is completed.

c) y incremented by 1.

d) Calculate waiting time and turnaround time

waiting time = Turnaround time - burst time

Turnaround time = completion time - arrival time

e) if $y == n$

break from the loop.

6.1.3 if $i == n-1$

set $i = 0$

6.1.4 else

Increment i by 1.

Step 7: Calculate average turnaround time and average waiting time.

Step 8: Display the process id, burst time, waiting time, turnaround time and average waiting time and average turnaround time.

Step 9: Stop.

RESULT

The program was executed successfully

and output was verified.

By
14/11/24

Experiment No. 5

INTERPROCESS COMMUNICATION.Aim

To write the program to perform interprocess communication.

ALGORITHM

Step 1: Start.

Step 2: Create variable shared memory, i , shm_id

Step 3: Assign shared memory segment created by $shmget()$ to shm_id

Step 4: display shm_id

Step 5: Assign shared memory with the address space of the process using $shmat()$

Step 6: Write the data into shared memory

Step 7: Create another program with same variables.

Step 8: Assign shared memory segment created by $shmget()$ to shm_id where $shmget()$ has same parameters as the previous one.

Step 9: Display shm_id

Step 10: Read shared memory with the address space of the process using $shmat()$

Step 11: Display address and data

is the Shared memory.
Step 12: Stop.

RESULT

The program was executed and output was verified.

✓
By 11/4/24

Experiment NO : 6.

MEMORY ALLOCATION METHODS

FIRST FIT

AIM

To write a C program to implement First Fit memory allocation.

ALGORITHM

Step 1: Start.

Step 2: Declare Structure process with variables pid, psize, alloc, bnum.

Step 3: Declare Structure block with variables bnum, bsize.

Step 4: Let $i=0, j=0$

Step 5: Read number of memory blocks n

Step 6: Read each memory block number and size.

Step 7: Read number of processes m .

Step 8: Read each process id and size and initially allocate process to 0.

Step 9: While $i < n$

 a.1 while $j < m$

 a) if $p[i].alloc = 0$ and
 $b[j].blockSize \geq p[i].psize$

b) Allocate $p[i].alloc = 1$

c) $p[i].bnum = b[j].bnum$

d) $b[j].bSize = b[j].bSize - p[i].pSize$

e) Exit from the loop

Step 10: print each process $pid, pSize, bnum$

Step 11: Stop.

RESULT

The program was executed and output was verified.

Page 11/14/24

BEST FIT

Aim

To write a C program to implement best fit memory allocation.

ALGORITHM

Step 1: Start

Step 2: Declare structure process with variables pid, psize, alloc, bnum

Step 3: Declare structure block with variables bnum, bsize.

Step 4: Let $i=0, j=0$

Step 5: Read number of memory blocks m .

Step 6: Read each memory block number & size.

Step 7: Read number of processes n

Step 8: Read each process id and size and initially allocate process to 0.

Step 9: While $i < n$

9.1 Sort the memory block in ascending order by bubble sort.

9.2 While $j < m$

a) if $p[i].alloc = 0$ and $b[j].block\ size \geq p[i].psize$

b) Allocate $p[i].alloc = 1$

c) $p[i].bnum = b[j].bnum$

d) $b[j].bsize = b[j].bsize - p[i].size$

e) Exit from the loop

Step 10: print each process pid, psize, hnum
Step 11: Stop

RESULT

The program was executed and output was verified.

WORST FIT

AIM

To write a C program to implement worst fit memory allocation.

ALGORITHM

Step 1: Start

Step 2: Declare structure process with variables
pid, psize, alloc, bnum.

Step 3: Declare structure block with variables
bnum, bsize

Step 4: Let $i = 0, j = 0$

Step 5: Read number of memory blocks m

Step 6: Read each memory block number
and size.

Step 7: Read number of process n .

Step 8: Read each process id and size and
initially allocate process to 0.

Step 9: While $i < n$

9.1 Sort the memory blocks in
descending order.

9.2 while $j < m$

a) if $p[i].alloc = 0$ and
 $b[j].block\ size \geq p[i].psize$

b) Allocate $p[i].alloc = 1$

c) $p[i].bnum = b[j].bnum$

d) $b[j].bsize = b[j].bsize - p[i].psize$.

c) Exit From loop

Step 10: print each process pid, psize, bnum

Step 11: stop

RESULT

The program was executed and output was verified.